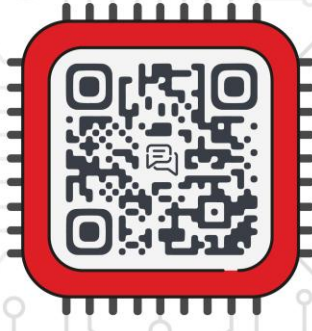




2



22



3300 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

تمارين داعمة لأفكار المحاضرة السابقة

عملي مشترك



RB Informatics; 5/5/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

مرحباً يا أصدقاء ، عدنا من جديد مع محاضرة جديدة ، هذه المحاضرة ستحتوي على تمارين داعمة لأفكار المحاضرة السابقة

التمرين الأول : حساب عدد العقد الكلي Total nodes

فكرة الحل : لحساب العدد الكلي لعقد شجرة ثنائية، نعتمد على **المرور العودي** على جميع العقد. الفكرة الأساسية هي أن كل عقدة تُحسب **كواحدة**، ثم نضيف إليها عدد العقد الموجودة في الجهة اليسرى وعدد العقد الموجودة في الجهة اليمنى. نبدأ بفحص الجذر، ثم نتنقل بشكل عودي إلى اليسار ثم إلى اليمين، ونجمع النتائج

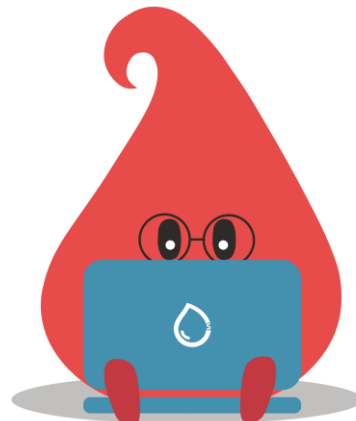
نستخدم في هذا الحل **العودية** لأنها تسمح لنا بالتعامل مع كل شجرة جزئية بنفس الطريقة تماماً، فكل عقدة تُعامل كأنها جذر لشجرة جديدة.

أما شرط التوقف فهو عندما نصل إلى عقدة فارغة (null)، وعندها نعيد القيمة 0 لأنها لا تحتوي على أي عقدة.

بعد ذلك، في كل استدعاء، نعيد:

1 + عدد العقد في اليسار + عدد العقد في اليمين

وبذلك نضمن أننا قمنا بعدّ جميع العقد في الشجرة مهما كان شكلها أو عمقها.

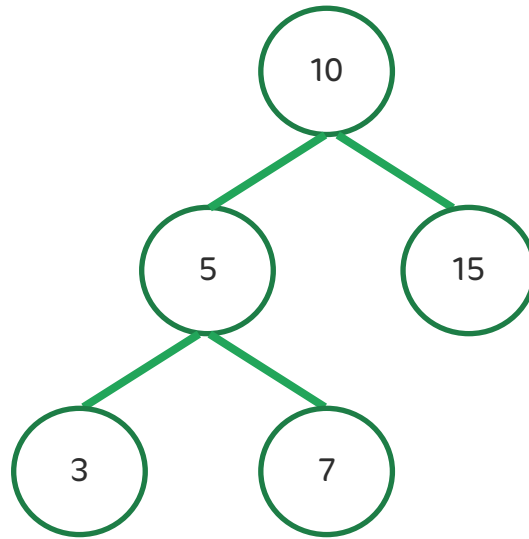


```
1 class Node {
2     int data; // usage
3     Node left, right; // 10 usages
4
5     Node(int value) { // 10 usages
6         data = value;
7         left = right = null;
8     }
9 }
10
11 class BinaryTree {
12     int totalNodes(Node root) { // 4 usages
13         if (root == null)
14             return 0;
15
16         return 1 + totalNodes(root.left) + totalNodes(root.right);
17     }
18 }
```

```
1 public static void main(String[] args) {
2     BinaryTree tree = new BinaryTree();
3     Node root = new Node( value: 10 );
4     root.left = new Node( value: 5 );
5     root.right = new Node( value: 15 );
6     root.left.left = new Node( value: 3 );
7     root.left.right = new Node( value: 7 );
8
9     System.out.println("Total Nodes = " + tree.totalNodes(root));
10 }
```

خرج الكود Total Nodes=5

توضيح شكل الشجرة:



التمرين الثاني : حساب عدد الأوراق في الشجرة

ملاحظة : الأوراق (Leaves) هي العقد التي ليس لديها أبناء

فكرة الحل: عند كل عقدة نجمع جميع العقد الورقية يمينها وجميع العقد الورقية يسارها ، نحلها بشكل عودي ، شرط التوقف: يوجد شرطان ، عند الوصول لعقدة null نرجع 0 وعند الوصول لعقدة ليس لها أبناء نرجع 1 (هذه ورقة)



```
1 class Node { 8 usages
2   int data; 1 usage
3   Node left, right; 7 usages
4
5   Node(int value) { 5 usages
6     data = value;
7     left = right = null;
8   }
9 }
10 class BinaryTree {
11   int countLeaf(Node root) { 3 usages
12     if (root == null)
13       return 0;
14
15     if (root.left == null && root.right == null)
16       return 1;
17
18     return countLeaf(root.left) + countLeaf(root.right);
19   }
20
21   public static void main(String[] args) {
22     BinaryTree tree = new BinaryTree();
23
24     Node root = new Node( value: 10 );
25     root.left = new Node( value: 5 );
26     root.right = new Node( value: 15 );
27     root.left.left = new Node( value: 3 );
28     root.left.right = new Node( value: 7 );
29
30     System.out.println("Leaf Nodes = " + tree.countLeaf(root));}
```

خرج الكود Leaf Nodes=3

توضيح:

الشجرة هي ذاتها في التمرين الاول
و العقد التي ليس لها أبناء هي (3 , 7 , 15)

التمرين الثالث: حساب عدد العقد غير الورقية (عدد العقد التي لديها ابن واحد على الأقل)

فكرة الحل :

لحساب عدد هذه العقد في الشجرة الثنائية، نقوم بفحص جميع العقد:

إذا كانت العقدة فارغة (null) نعيد القيمة 0.

إذا كانت العقدة لا تملك أبناء فهي عقدة ورقية، لذلك لا تُحسب.

إذا كانت العقدة تملك ابناً واحداً على الأقل، فإنها تُعد عقدة غير ورقية، لذلك نضيف 1 ثم نكمل الفحص في الفرعين الأيسر والأيمن.

```

2 class Node { 8 usages
3     int data; 1 usage
4     Node left, right; 7 usages
5
6     Node(int value) { 5 usages
7         data = value;
8         left = right = null;
9     }
10 }
11 class BinaryTree {
12     int countNotLeaf(Node root) { 3 usages
13         if (root == null)
14             return 0;
15
16         if (root.left == null && root.right == null)
17             return 0;
18
19         return 1 + countNotLeaf(root.left) + countNotLeaf(root.right);
20     }
21
22     public static void main(String[] args) {
23         BinaryTree tree = new BinaryTree();
24
25         Node root = new Node( value: 10 );
26         root.left = new Node( value: 5 );
27         root.right = new Node( value: 15 );
28         root.left.left = new Node( value: 3 );
29         root.left.right = new Node( value: 7 );
30
31         System.out.println("Non-Leaf Nodes = " + tree.countNotLeaf(root));
32     }

```

خرج الكود:

Non-Leaf Nodes = 2

توضيح: الشجرة هي ذاتها في التمرينين السابقين لكن العقد التي تمتلك ابن واحد على الأقل هي (5, 10)

التمرين الرابع : حساب ارتفاع الشجرة (Height)

ملاحظة : ارتفاع الشجرة هو أطول مسار من الجذر الى أي ورقة

فكرة الحل : نحلها بشكل عودي، فنقوم بالخطوات التالية:

إذا كانت العقدة فارغة (null) يكون الارتفاع (-1)

نحسب ارتفاع الفرع الأيسر.

نحسب ارتفاع الفرع الأيمن.

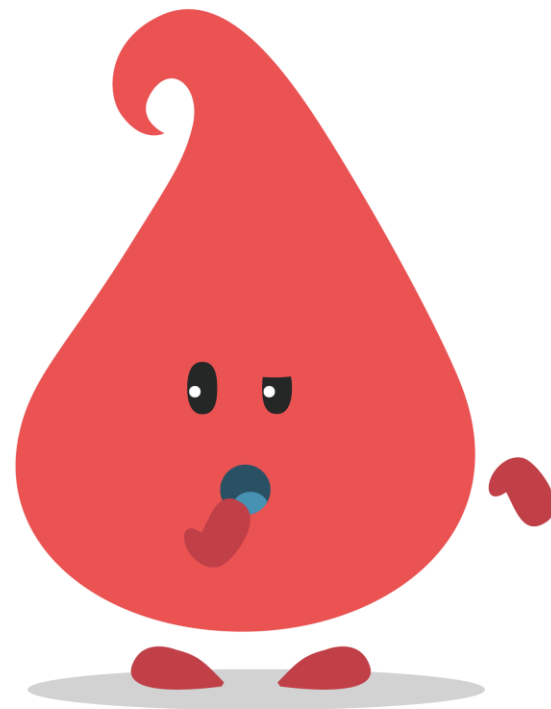
أخذ الحد الأقصى بينهما ثم نضيف 1 لتمثيل عدد الوصلات

وبذلك يكون: ارتفاع الشجرة = $1 + \text{أكبر ارتفاع بين الفرع الأيسر والفرع الأيمن}$.

```

2 class Node {
3     int data; // usage
4     Node left, right; // 5 usages
5
6     Node(int value) { // 4 usages
7         data = value;
8         left = right = null;
9     }
10 }
11
12 class BinaryTree {
13     int height(Node root) { // 3 usages
14         if (root == null)
15             return -1;
16
17         int leftHeight = height(root.left);
18         int rightHeight = height(root.right);
19
20         return Math.max(leftHeight, rightHeight) + 1;
21     }
22
23     public static void main(String[] args) {
24         BinaryTree tree = new BinaryTree();
25
26         Node root = new Node(10);
27         root.left = new Node(5);
28         root.right = new Node(15);
29         root.left.left = new Node(3);
30
31         System.out.println("Height of Tree = " + tree.height(root));
32     }

```

خرج الكود :Height of Tree = 2

ملاحظة بسيطة: في الشجرة الموجودة بالمثل:

الارتفاع عند العقدة 3 هو 0.

الارتفاع عند العقدة 5 هو 1.

الارتفاع عند الجذر 10 هو 2.

لذلك النتيجة التي ستظهر لك هي 2.

ملاحظة اربيسيزية :

Math.max هي دالة (Method) جاهزة موجودة بلغة جافا، ومهمتها بسيطة جداً: المقارنة بين رقمين وإعطاؤك الرقم الأكبر بينهما. واستخدمناها هنا لنختار أطول مسار ممكن من الجذر إلى أبعد ورقة.

التمرين الخامس : حساب عدد العقد بكل مستوى من الشجرة

فكرة الحل : لحساب عدد العقد الموجودة في كل مستوى من مستويات الشجرة، نعتمد على أسلوب المرور بالمستويات (Level Order Traversal)، حيث تتم زيارة العقد مستوى بعد مستوى بدءاً من الجذر.

نستخدم Queue لتخزين العقد التي يجب المرور عليها لاحقاً. في البداية نضع عقدة الجذر داخل الـ Queue، ثم في كل دورة نحدد عدد العقد الموجودة داخله، وهذا العدد يمثل عدد عقد المستوى الحالي.

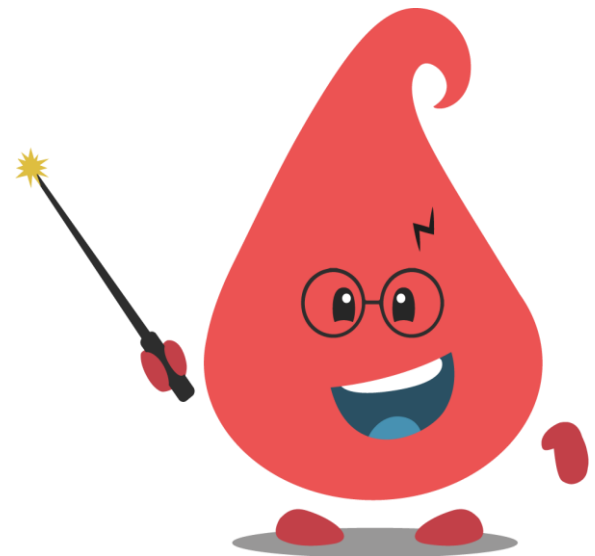
بعد ذلك نقوم بإخراج هذه العقد واحدة تلو الأخرى، وعند كل عقدة نضيف أبنائها (الأيسر ثم الأيمن إن وُجدوا) إلى الـ Queue، ليتم التعامل معهم في المستوى التالي.

نكرر هذه الخطوات حتى يصبح الـ Queue فارغاً، وبذلك نكون قد حسبنا عدد العقد في جميع مستويات الشجرة.

```
1. import java.util.LinkedList;
2. import java.util.Queue;
3. class BinaryTree {
4.     Node root;
5.     class Node {
6.         int value;
7.         Node left, right;
8.         Node(int value) {
9.             this.value = value;
10.            this.left = this.right = null;
11.        }
12.    }
13. }
```

```
1
2 public void countNodesAtEachLevel
3 (Node root) {
4     if (root == null) return;
5     int level = 1;
6     Queue<Node> queue = new LinkedList<>();
7     queue.add(root);
8     while (!queue.isEmpty()) {
9         int cnt = queue.size();
10        System.out.print("Level " + level++ + ": " + cnt);
11        for (int i = 0; i < cnt; i++) {
12            Node node = queue.poll();
13            if (node.left != null)
14                queue.add(node.left);
15            if (node.right != null)
16                queue.add(node.right);
17        }
18        System.out.println();
19    }
20 }
```

```
1 public class Main {
2     public static void main(String[] args){
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(10);
5         bt.root.left = bt.new Node(20);
6         bt.root.left.left = bt.new Node(50);
7         bt.root.left.right = bt.new Node(40);
8         bt.root.right = bt.new Node(39);
9         bt.root.right.right = bt.new Node(2);
10        bt.root.right.right.right = bt.new Node(5);
11        bt.countNodesAtEachLevel(bt.root);
12
13        // Level 1: 1 // Level 2: 2
14        // Level 3: 3 // Level 4: 1
15    }
16 }
```



Level 1

1

10

Level 2

1

20

2

30

Level 3

1

50

2

40

3

2

Level 4

1

5

التمرين السادس : حساب عدد العقد في مستوى معين من الشجرة

فكرة الحل : لإيجاد عدد العقد الموجودة في مستوى معين داخل الشجرة، نمرر رقم المستوى المطلوب إلى الدالة مع عقدة الجذر.

إذا كانت العقدة الحالية فارغة فهذا يعني عدم وجود عقد في هذا الفرع، لذلك نعيد القيمة 0. أما إذا أصبح رقم المستوى يساوي 1 فهذا يعني أننا وصلنا إلى المستوى المطلوب، وبالتالي نعيد 1 لاحتساب هذه العقدة. في غير ذلك، ننتقل إلى الأبناء الأيسر والأيمن مع إنقاص رقم المستوى بمقدار واحد، لأننا نتحرك خطوة أعمق داخل الشجرة. ثم نجمع النتائج القادمة من الفرعين للحصول على العدد الكلي للعقد في ذلك المستوى.

```

1 import java.util.LinkedList;
2 import java.util.Queue;
3
4
5 class BinaryTree {
6     Node root;
7     class Node {
8         int value;
9         Node left, right;
10        Node(int value) {
11            this.value = value;
12        }
13        this.left = this.right = null;
14    }
15 }

```

```
1 public int countNodesAtCertainLevel(Node root, int level) {  
2     if (root == null) return 0;  
3     if (level == 1) return 1;  
4     return countNodesAtCertainLevel(root.left, level - 1) + countNodesAtCertainLevel(root.right,  
level - 1);  
5 }  
6 }
```

```
2 public class Main {  
3     public static void main(String[] args) {  
4         BinaryTree bt = new BinaryTree();  
5         bt.root = bt.new Node(10);  
6         bt.root.left = bt.new Node(20);  
7         bt.root.left.left = bt.new Node(50);  
8         bt.root.left.right = bt.new Node(40);  
9         bt.root.right = bt.new Node(39);  
10        bt.root.right.right = bt.new Node(2);  
11        bt.root.right.right.right = bt.new Node(5);  
12        System.out.println(bt.countNodesAtCertainLevel(bt.root,3)); // 3  
13    }  
14 }
```

التمرين السابع : حساب عرض الشجرة (The maximum number of nodes at any level in the tree)

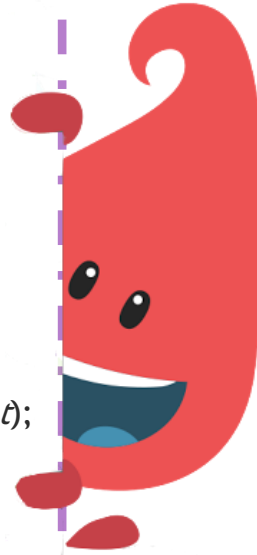
فكرته مشابهة لفكرة التمرين الخامس لكن هذه المرة بدلا من عد العقد لكل مستوى سنقوم فقط باختيار عدد العقد الاعظمي

```
1 import java.util.LinkedList;  
2 import java.util.Queue;  
3 class BinaryTree {  
4     Node root;  
5     class Node {  
6         int value;  
7         Node left, right;  
8         Node(int value) {  
9             this.value = value;  
10            this.left = this.right = null;  
11        }  
12    }
```

```

1 public int treeWidth(Node root) {
2     if (root == null) return 0;
3     Queue<Node> queue = new LinkedList<>();
4     queue.add(root);
5     int max = 0;
6     while (!queue.isEmpty()) {
7         int cnt = queue.size();
8         max = Math.max(max, cnt);
9         for (int i = 0; i < cnt; i++) {
10             Node node = queue.poll();
11             if (node.left == null) queue.add(node.left);
12             if (node.right == null) queue.add(node.right);
13         }
14     }
15     return max;
16 }

```



```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(10);
5         bt.root.left = bt.new Node(20);
6         bt.root.left.left = bt.new Node(50);
7         bt.root.left.right = bt.new Node(40);
8         bt.root.right = bt.new Node(39);
9         bt.root.right.right = bt.new Node(2);
10        bt.root.right.right.right = bt.new Node(5);
11        System.out.println(bt.treeWidth(bt.root)); // 3
12    }
13 }

```

التمرين الثامن : البحث عن اكبر عنصر في شجرة ثنائية

فكرة الحل: لإيجاد أكبر عنصر في الشجرة الثنائية، نقوم بزيارة جميع العقد الموجودة في الشجرة لأن القيمة الأكبر يمكن أن تكون في أي مكان داخلها، وليس في موقع محدد.

نبدأ من الجذر، ثم ننتقل بشكل متكرر إلى الفرع الأيسر والفرع الأيمن بشكل عودي

في كل خطوة نقارن قيمة العقدة الحالية مع أكبر قيمة تم إيجادها في الفرعين، ونحتفظ بالقيمة الأكبر بينهما.

وفي النهاية، تكون النتيجة هي أكبر قيمة موجودة في جميع عقد الشجرة

```

1 class BinaryTree{
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             left = right = null;
9         }
10    };
11 };

```

```

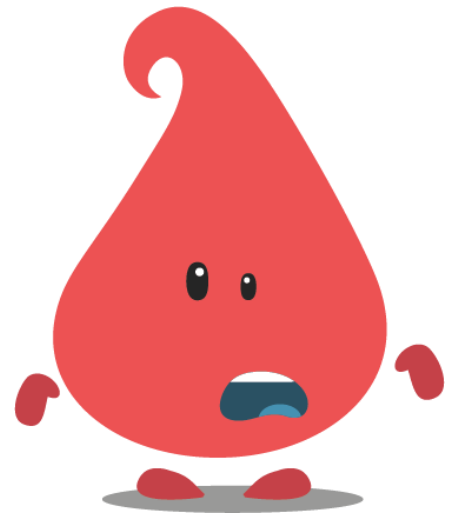
1 public static int findMax(Node root) {
2     if (root == null)
3         return Integer.MIN_VALUE;
4     int maxLeft = findMax(root.left);
5     int maxRight = findMax(root.right);
6     return Math.max(maxRoot.value, Math.max(maxLeft, maxRight));
7 }
8 }

```

```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(10);
5         bt.root.left = bt.new Node(20);
6         bt.root.left.left = bt.new Node(50);
7         bt.root.left.right = bt.new Node(40);
8         bt.root.right = bt.new Node(39);
9         bt.root.right.right = bt.new Node(2);
10        int max = bt.findMax(bt.root);
11        System.out.println("the max value is : " + max);
12    }
13 }

```



التمرين التاسع : عكس العقد في شجرة

فكرة الحل: لعكس شجرة ثنائية نحلها بشكل عودي بحيث نقوم بزيارة كل عقدة في الشجرة بشكل متكرر حتى نصل إلى جميع العقد. تبدأ العملية من العقدة الجذر، فإذا كانت العقدة فارغة نوقف التنفيذ (حالة الأساس). بعد ذلك نقوم باستدعاء الدالة نفسها على الفرع الأيسر ثم على الفرع الأيمن لضمان الوصول إلى جميع العقد في الشجرة. بعد الانتهاء من معالجة الفروع، نقوم في كل عقدة بتبديل الابن الأيسر مع الابن الأيمن باستخدام متغير مؤقت، بحيث يصبح الفرع الأيسر مكان الفرع الأيمن والعكس. تستمر هذه العملية لكل العقد في الشجرة حتى نصل إلى الجذر مرة أخرى، وبذلك نحصل على شجرة مقلوبة بالكامل تعكس اتجاه جميع الفروع.

```
1 class BinaryTree{
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             left = right = null;
9         }
10    }
```

```
1 public void invertTree(Node root) {
2     if (root == null)
3         return;
4     // مؤقت متغير تتطلب التبديل عملية
5     Node temp = root.left;
6     root.left = root.right;
7     root.right = temp;
8     invertTree(root.left);
9     invertTree(root.right);
10 }
```

```
1 public void inorderTraversal(Node root) {
2     if (root!=null) {
3         inorderTraversal(root.left);
4         System.out.print(root.value + " ");
5         inorderTraversal(root.right);
6     }
7 }
8 }}
```

```
1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(10);
5         bt.root.left = bt.new Node(20);
6         bt.root.left.left = bt.new Node(50);
7         bt.root.left.right = bt.new Node(40);
8         bt.root.right = bt.new Node(39);
9         bt.root.right.right = bt.new Node(2);
10
11         System.out.println("in-order TRAVERSAL before inverting :");
12         bt.inorderTraversal(bt.root);
13         bt.invertTree(bt.root);
14         System.out.println();
15         System.out.println();
16         System.out.println("in-order TRAVERSAL after inverting :");
17         bt.inorderTraversal(bt.root);
18     }
19 }
```

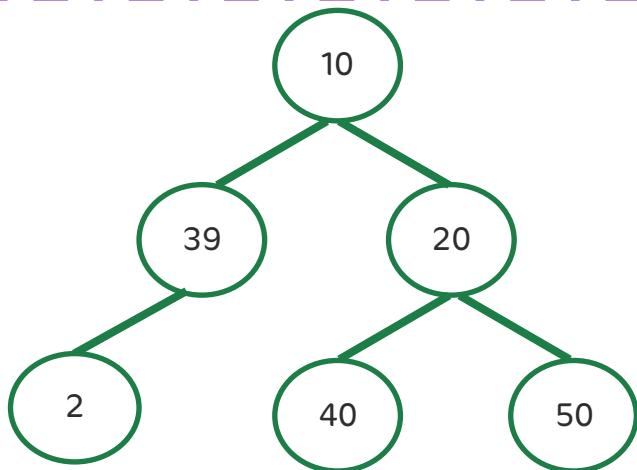
في هذه الشجرة الخرج سيكون :

In-order TRAVERSAL before inverting:

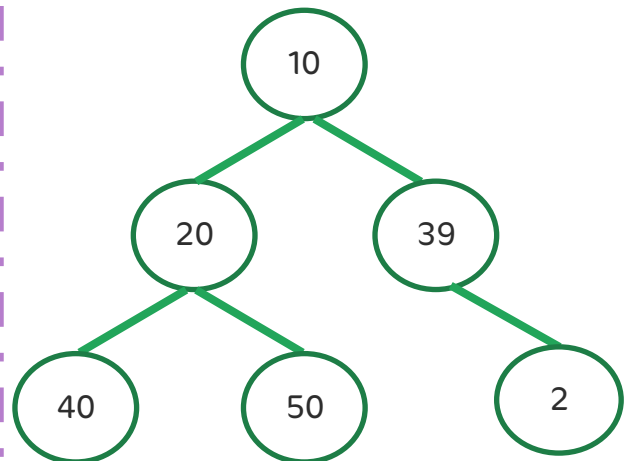
50 20 40 10 39 2

in-order TRAVERSAL after inverting :

2 39 10 40 20 50



AFTER INVERTING



BEFOR INVERTING

التمرين العاشر: هل الشجرة المعطاة خطية ؟

■ متى نقول انها خطية ؟ اذا كانت كل عقدة تمتلك ابن واحد على الأكثر (اما ابن او ولا ابن)

فكرة الحل : عند فحص الشجرة الثنائية لمعرفة ما إذا كانت خطية، ننظر إلى كل عقدة ونتحقق من عدد الأبناء لديها. الفكرة الأساسية أن الشجرة تُعد خطية عندما لا تمتلك أي عقدة أكثر من ابن واحد، بحيث يصبح مسارها ممتدًا في اتجاه واحد يشبه تمامًا طريقة عمل الـ Linked List؛ فكما أن كل عنصر في الـ لينكد ليست يشير إلى عنصر واحد فقط، فإن الشجرة الخطية تسير في مسار واحد دون أي تفرع. يعتمد هذا الفحص على الاستدعاء العودي؛ فعند كل عقدة نتقل إلى ابنها الوحيد (إن وُجد) ونكرر نفس التحقق على طول المسار. أما إذا صادفنا عقدة لها ابنان معًا، فهذا يعني أن الشجرة فقدت شكلها الخطي. وفي حال كانت العقدة بلا أبناء، فهذا يمثل نهاية طبيعية للمسار. كما أن الوصول إلى عقدة فارغة (null) يُعد حالة صحيحة لأنه يشير إلى نهاية فرع ولا يؤثر على النتيجة. بهذه الطريقة نضمن المرور على كامل الشجرة باستخدام العودية والتأكد من مطابقة جميع العقد لشرط الخطية.

```
1 import java.util.LinkedList;
2 import java.util.Queue;
3 class BinaryTree {
4     Node root;
5     class Node {
6         int value;
7         Node left, right;
8         Node(int value) {
9             this.value = value;
10            this.left = this.right = null; }
11     }
```

```
1 public boolean isLinearTree(Node root) {
2     if (root == null) return true;
3     if ((root.left == null && root.right != null) ||
4         (root.left != null && root.right == null))
5         return isLinearTree(root.left) &&
6             isLinearTree(root.right);
7     return root.left == null && root.right == null;
8 }
9 }
```

```
1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(10);
5         bt.root.left = bt.new Node(20);
6         bt.root.left.left = bt.new Node(50);
7         bt.root.left.right = bt.new Node(40);
8         bt.root.right = bt.new Node(39);
9         bt.root.right.right = bt.new Node(2);
10        bt.root.right.right.right = bt.new Node(5);
11        System.out.println(bt.isLinearTree(bt.root)); // false
12    }
13 }
```

التمرين الحادي عشر : هل الشجرة المعطاة مكتملة ؟

ملاحظة : الشجرة المكتملة هي شجرة تكون فيها كل المستويات ممتلئة، وإذا كان هناك نقص في آخر مستوى، فيجب أن يكون النقص من الجهة اليمنى فقط.

فكرة الحل :

حتى نتحقق مما إذا كانت الشجرة الثنائية مكتملة، نستخدم أسلوب ترقيم العقد اعتماداً على طريقة الـ *indexing* *method*. نبدأ من العقدة الجذر و نعطيها الرقم 1، ثم يصبح رقم الابن الأيسر هو ($2*i$) ورقم الابن الأيمن هو ($2*i+1$). هذا الترقيم يساعدنا على معرفة الرقم الذي يُفترض أن تصل إليه أي عقدة ضمن شجرة مكتملة.

وبما أننا تعلمنا في التمرين الاول كيف نحسب العدد الكلي للعقد، يمكننا استخدام هذا العدد لمعرفة الحد الأعلى لـ *index* الذي يمكن أن تظهر فيه عقدة داخل شجرة مكتملة. فمثلاً، إذا كان لدينا 7 عقد، فهذا يعني أن أكبر رقم يمكن أن تصل إليه أي عقدة هو 7، وأي عقدة تحمل رقماً أكبر من ذلك تكون خارج المجال المتوقع.

نقوم بمتابعة رقم كل عقدة أثناء المرور عليها باستخدام العودية، ونتحقق في كل خطوة مما إذا كان رقمها يقع ضمن الحدود الصحيحة. أما شروط التوقف فهي كالتالي :

إذا وصلنا لعقدة فارغة (null) نرجع true لأنها لا تؤثر على الفحص.

وإذا كان رقم العقدة أكبر من العدد الكلي للعقد، نرجع false لأنها تجاوزت الحد الذي يمكن أن تصل إليه عقدة في شجرة مكتملة.

```

1 class BinaryTree {
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             this.left = this.right = null; }}
9     public boolean isCompleteTree(Node root, int index, int last) {
10        if (root == null) return true;
11        if (index > last) return false;
12        return isCompleteTree(root.left, 2 * index, last) &&
13        isCompleteTree(root.right, 2 * index + 1, last); }
14    public int size(Node root) {
15        if (root == null) return 0;
16        return 1 + size(root.left) + size(root.right); }}
17 }
```



```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(10);
5         bt.root.left = bt.new Node(20);
6         bt.root.left.left = bt.new Node(50);
7         bt.root.left.right = bt.new Node(40);
8         bt.root.right = bt.new Node(39);
9         bt.root.right.right = bt.new Node(2);
10        bt.root.right.right.right = bt.new Node(5);
11        System.out.println(bt.isCompleteTree(bt.root,1, bt.size(bt.root))); //false
12        bt.root.right.right.right = null;
13        bt.root.right.right = null;
14        bt.root.right.left = bt.new Node(5);
15        System.out.println(bt.isCompleteTree(bt.root,1, bt.size(bt.root))); //true
16    }}

```

التمرين الثاني عشر: تمثيل معادلة رياضية بشجرة وطباعة الناتج

فكرة الحل : لتمثيل المعادلة الرياضية على شكل شجرة نلتزم بعدة قواعد أساسية.

أولاً: أي عقدة تمتلك أبناء يجب أن تكون عملية رياضية، بينما الأبناء يجب أن يكونوا قيماً عددية. كذلك، لا يمكن أن تحتوي أي عقدة على ابن واحد فقط، فإما أن يكون لها ابنان أو لا أبناء إطلاقاً.

أما العقدة التي لا تمتلك أبناء، فيجب أن تكون قيمة رقمية، وتتحقق من هذا الشرط في كل شجرة جزئية أثناء عملية الفحص العودي. عند الوصول إلى الأوراق سنجد قيماً عددية فقط، وبعدها نبدأ بالاستدعاء العودي:

لكل عقدة، نحسب قيمة الشجرة الموجودة على يسارها، ثم قيمة الشجرة الموجودة على يمينها، وبعد ذلك نطبق العملية الموجودة في هذه العقدة على القيمتين الناتجتين.

شرط التوقف هو الوصول إلى ورقة لا تحتوي على عملية، وعندها نعيد القيمة العددية مباشرة.

```

1 class BinaryTree {
2     Node root;
3     class Node {
4         String value;
5         Node left, right;
6         Node(String value) {
7             this.value = value;
8             this.left = this.right = null;
9         }
10    }
11 }

```

```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         //((2+4)*3)
5         bt.root = bt.new Node("*");
6         bt.root.left = bt.new Node("+");
7         bt.root.right = bt.new Node("3");
8         bt.root.left.left = bt.new Node("2");
9         bt.root.left.right = bt.new Node("4");
10        int result = bt.equation(bt.root);
11        System.out.println("result: " + result);
12        // 18
13    }
14 }

```

```

1 public int equation(Node root) {
2     if (root.left == null && root.right == null)
3         return Integer.parseInt(root.value);
4     int leftVal = equation(root.left);
5     int rightVal = equation(root.right);
6     switch (root.value) {
7         case "+":
8             return leftVal + rightVal;
9         case "-":
10            return leftVal - rightVal;
11        case "*":
12            return leftVal * rightVal;
13        case "/":
14            return leftVal / rightVal;
15        default:
16            System.out.println("invalid
17 operator: " + root.value);
18        return 0;
19    }
20 }

```

التمرين الثالث عشر : لدينا مصفوفتان يعبران عن preorder and inorder representation of a tree

■ مهمتك هي إعادة بناء هذه الشجرة بناء على هذين التمثيلين.

فكرة الحل : لدينا المصفوفة تمثل الـ preorder traversal و الذي من خلاله كنا نعالج الجذر أولاً ثم نذهب إلى الطرف الأيسر ثم الطرف الأيمن و المصفوفة الأخرى تمثل الـ inorder traversal التي نعالج من خلالها الطرف الأيسر ثم الجذر ثم الطرف الأيمن حتى نبني الشجرة الأصلية بشكلها المطلوب سنعتمد على نقطتين: نحن نعرف أن العقدة الأولى في مصفوفة preorder تمثل الجذر الأول و الذي سنتطلق منه كونه الجذر نضيفه كعقدة أولى للشجرة

و بعد ذلك سنبحث عن هذا الجذر في مصفوفة inorder و الهدف من ذلك أن نعرف ما هي عناصر الشجرة اليسرى و ما هي عناصر الشجرة اليمنى بحيث يمكننا حصر المجالات فكل العناصر قبل الجذر في مصفوفة inorder تقع بالشجرة اليسرى و كل العناصر بعد الجذر في مصفوفة inorder تقع بالشجرة اليمنى و بالتالي تستدعي التابع بشكل عودي على الشجرة اليمنى والشجرة اليسرى و نقوم بنفس الموضوع بالنسبة لكل عقدة .

بحيث تعتبرها جذر الشجرة الجديدة و كل العناصر على يمينها ضمن المجال الذي حددناه سابقاً تقع على يمينها وكل العناصر على يسارها ضمن المجال الذي حددناه سابقاً تقع على يسارها و في كل استدعاء نضيف العقدة التالية ضمن مصفوفة preorder و نظراً لأن ترتيب سير المصفوفة هذه هو :

(جذر - يسار - يمين) سنقوم ببناء الشجرة اليسرى أولاً ثم بناء الشجرة اليمنى ثانياً بطريقة عودية (هكذا نضمن أن إضافة العقدة التالية ضمن كل استدعاء عودي ستكون في محلها).

شروط التوقف :

عندما تصبح بداية المجال بعد نهاية المجال ترجع null (لم يعد هنالك عقد).

عندما تصبح بداية المجال تساوي نهاية المجال ترجع العقدة الحالية التي نقف عندها دون استدعاءات إضافية .

بعد بناء الشجرة نقوم بتعيين الجذر بإرجاع الـ root في النهاية.



```
1 class BinaryTree {
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             this.left = this.right = null; }
9         int preIndex;
10        public Node buildTree(int[] inorder, int[] preorder) {
11            preIndex = 0;
12            return build(inorder, preorder, 0, inorder.length-1);
13        }
14        private Node build(int[] inorder, int[] preorder, int l, int r) {
15            if (l > r)
16                return null;
17            Node node = new Node(preorder[preIndex]);
18            preIndex++;
19            if (l == r)
20                return node;
21            int s = search(inorder, l, r, node.value);
22            node.left = build(inorder, preorder, l, s - 1);
23            node.right = build(inorder, preorder, s + 1, r);
24            return node;
25        }
26        private int search(int[] arr, int l, int r, int val) {
27            for (int i = l; i <= r; i++)
28                if (arr[i] == val)
29                    return i;
30            return -1; }
31    }
32 }
```

```

1  public class Main {
2      public static void main(String[] args) {
3          BinaryTree bt = new BinaryTree();
4          //      10
5          //    /  \
6          //   20   39
7          //  / \   \
8          // 50 40   2
9          //         \
10         //          5
11         int[] preorder = {10,20,50,40,39,2,5};
12         int[] inorder = {50,20,40,10,39,2,5};
13         bt.root = bt.buildTree(inorder, preorder);
14         System.out.println(" " + bt.root.value);
15         System.out.println(" " + bt.root.left.value + " " + bt.root.right.value);
16         System.out.println(bt.root.left.left.value + " " + bt.root.left.right.value + " " +
17         bt.root.right.right.value);
18         System.out.println(" " + bt.root.right.right.right.value); } }

```

التمرين الرابع عشر والأخير : حساب عدد العقد في شجرة مكتملة من خلال ارتفاعها

فكرة الحل : بما أن الشجرة ثنائية مكتملة، فهذا يعني أن جميع مستوياتها ممتلئة بالكامل، وكل مستوى يحتوي على ضعف عدد العقد الموجودة في المستوى الذي قبله. لذلك يمكننا حساب العدد الكلي للعقد اعتماداً على الارتفاع فقط، دون الحاجة للمرور على الشجرة أو استخدام العودية.

نعتمد على العلاقة الرياضية التي تعبر عن مجموع العقد في شجرة مكتملة، حيث يكون مجموع العقد في جميع المستويات من المستوى 0 حتى المستوى h مساوياً للقيمة:

$$(2^{h+1} - 1)$$

هذه الصيغة تجمع عدد العقد في كل المستويات دفعة واحدة، لأنها تعتمد على خاصية التضاعف في الشجرة المكتملة، وبذلك نحصل على عدد العقد مباشرة بمجرد معرفة الارتفاع.

```
1 class CompleteTree {
2     public int totalNodes(int height) { 1usage
3         // إذا كان الارتفاع ساكن فهذا يعني شجرة فارغة
4         if (height < 0)
5             return 0;
6         return (int)(Math.pow(2, height + 1) - 1);
7     }
8
9     public static void main(String[] args) {
10        CompleteTree tree = new CompleteTree();
11
12        int h = 3; // ارتفاع الشجرة
13        int result = tree.totalNodes(h);
14
15        // Total Nodes = 15: خرج الكود
16        System.out.println("Total Nodes = " + result);
17    }
18 }
```

وهكذا نكون قد وصلنا الى نهاية محاضرتنا ، نلتقاكم

في المحاضرة القادمة

